

Exact Bidirectional Steganographic Transport with Language and Pixel Autoregressive Models

Keywords: Generative Steganography, Autoregressive Models, Exact Recovery, Steganalysis, GPU Inference.

Abstract: We encode a small image within generated text and a text message within an image, allowing a recipient with the required key and model configuration to recover the original content. Encryption protects the content, while steganographic encoding places it within another medium. A language model assigns probabilities to text choices, and a pixel autoregressive model does the same for pixel values. Encoding rules use these choices to carry information, and the receiver reconstructs the mapping from the saved file. We develop a shared authenticated protocol for exact recovery of standardized grayscale image pixels and literal text from actual text and PNG files. An extension preserves an existing photograph through small pixel changes. Controlled evaluations measure recovery, capacity, detection and execution cost. Across 120 transmissions, fixed and entropy-gated coding each recover 20/20 payloads in both directions. Arithmetic coding recovers 20/20 PNG payloads but 0/20 text payloads because insufficient information accumulates before the length limit. The photograph extension recovers 20/20 held-out messages, but learned ranks do not outperform simple parity. Detection results depend on the observer’s context, and a matched development benchmark reduces encoding and decoding latency by 5.13 times using CUDA graphs.

1 INTRODUCTION

A sender may wish to send a small image through a text passage or place a short message within a photograph. The hidden content is the payload, and the visible passage or image is its carrier. A recipient with the required key and model configuration must recover the exact represented image pixels or message bytes from that carrier. Generative models make many plausible text or pixel choices available, but those choices must carry information in a way the recipient can reconstruct. Recovery from the saved file matters because the recipient receives the passage or pixels, not the sender’s internal decisions.

Calgacus proposes cross domain rank transfer, RankCloak develops bounded byte coding and entropy gating, and Pixel Stega carries bits through autoregressive images (Norelli and Bronstein, 2026; Mantzaris, 2026b; Zhang et al., 2021). We connect these foundations through a shared authenticated protocol for canonical images and literal text. Fresh receivers reconstruct state from saved carriers, and an independent evaluator verifies source equality. A controlled three-coder comparison exposes how modality, serialization and finite capacity interact. The construction progresses from image transport through generated text to text transport through generated PNGs, then to small changes in natural photographs. The photograph profile reconstructs model ranks from

invariant coarse pixels without access to the original cover. A paired parity baseline tests whether these learned partitions improve embedding. Declared observer detection and exact GPU replay comparisons complete the evaluation of recovery, observer knowledge and execution cost.

2 RELATED WORK

2.1 Distributional and Rank-Based Coding

Neural Linguistic Steganography (Ziegler et al., 2019) encodes a message through arithmetic decoding and recovers it by replaying arithmetic encoding. Our comparator follows that distribution aware approach with an explicit finite message adaptation. The coupling formulation (Schroeder de Witt et al., 2023) characterizes perfectly secure procedures, with minimum entropy coupling evaluated on language, audio and image channels. Rotation-based range coding (Yan and Murawaki, 2026) offers distribution preservation guarantees and high entropy utilization. These guarantees concern a different objective from our matched packet-recovery comparison.

Calgacus (Norelli and Bronstein, 2026) transfers a payload token’s context dependent rank to a cover context and discusses different models, vocabulary

sizes and discrete domains. RankCloak (Mantzaris, 2026b) implements bounded byte coding and entropy gating for serialized cryptographic artifacts. Its reported rendered text failures motivate our saved artifact endpoint. LlmStenoExplore (Mantzaris, 2026a) investigates finite key collisions and perturbations. We adapt RankCloak’s rank, replay and numerical practices, separating the packet key from model conditioning.

2.2 Pixels and Tokenization

Pixel-Stega (Zhang et al., 2021) is the closest image precedent. It applies finite precision arithmetic stegosampling to PixelCNN++ probabilities over a lossless channel. Its color formulation identifies red as the embedded variable. We evaluate successive observable R, G and B values with conditional mixture posteriors and a shared row canvas. Three coders share one authenticated contract across text and PNG. Pixel-Stega’s published capacities and detector scores use different allocations, not our matched net rates.

Published stepwise verification (Yan and Murawaki, 2025) addresses detokenization inconsistency by verifying candidates. We adapt this idea to byte exact tokenization and a top-256 pool. The development comparison isolates singleton versus complete prefix consistency with other rules fixed.

2.3 Detectability and Execution

GLTR (Gehrmann et al., 2019) exposes probability and rank statistics for generated-text analysis. Our observer instead compares steganographic carriers with ordinary samples of the same eligible distribution, using two frozen scores. The context experiment changes only its row while retaining the known image model. CUDA graphs are established infrastructure (PyTorch Contributors, 2024). We measure repeated PixelCNN++ replay while requiring exact coder-interface and artifact equivalence. Numerical changes could otherwise reorder ranks and break decoding.

2.4 Embedding in Existing Photographs

Quantization index modulation selects reconstruction sets to balance embedding distortion and robustness (Chen and Wornell, 2000). Syndrome trellis codes minimize additive embedding costs (Filler et al., 2011). HiDDeN and StegaStamp train encoders and decoders with distortion channels (Zhu et al., 2018; Tancik et al., 2020). Our photograph construction instead rebuilds pretrained model rank partitions within

an invariant coarse context. Nearest-candidate selection bounds distortion, and delivered pixels suffice to reconstruct the context. This connects the Calgacus-style ranks to cover modification without a trained embedding network. The measured comparator is simple RGB parity, not these established systems.

3 METHOD

We first represent a standardized source image or literal message as bytes and place them in a fixed size encrypted and authenticated packet. The chosen coding rule maps packet information to text choices made with a language model or pixel choices made with a pixel autoregressive model. The receiver reads the saved text or PNG and uses the key, shared context and model configuration to reconstruct and authenticate the packet. A separate evaluator compares the recovered bytes with the source to establish exact recovery. The photograph extension instead constrains changes to an existing image and rebuilds its rank mapping from the preserved image context.

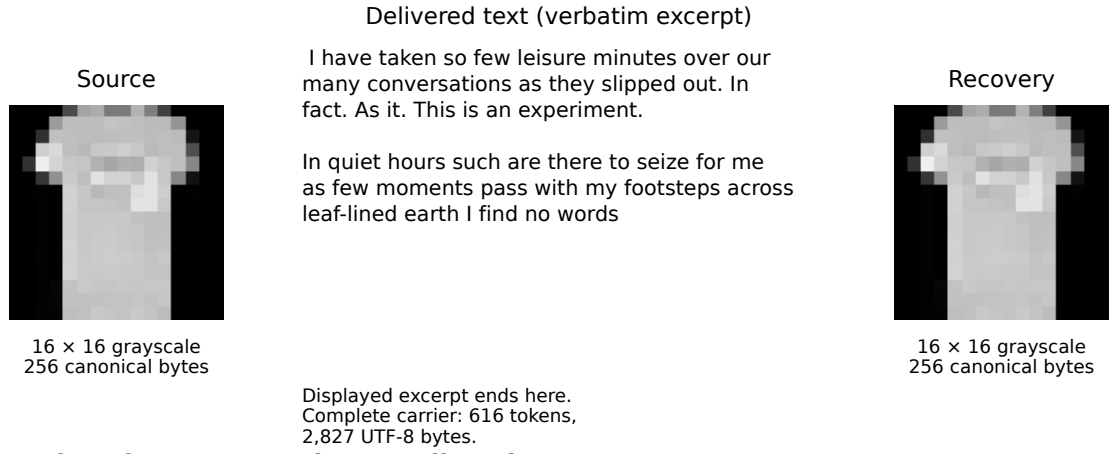
3.1 A Shared Artifact Contract

A public profile specifies the model, observable representation, numerical rules, coder and budget. The sender seals canonical source bytes once and encodes the packet into carrier y . The receiver obtains only saved y , the profile, shared context and encryption key. Source bytes/digests, sender identifiers, ranks, packets and latent codes are excluded. Figure 1 shows both directions using the first manifest-ordered fixed-rank example in each direction. Panel A carries canonical image pixels through saved text. Panel B carries literal text through a saved PNG.

Images are canonical 16×16 grayscale arrays of unsigned eight bit pixels, 256 bytes in row major order. Exact recovery restores this preprocessed array, not the larger original image or its container. Text is 32-128 literal UTF-8 bytes without normalization, added newlines or replacement decoding. The plain-text header contains version (one byte, 1), kind (one byte, 1 for text or 2 for grayscale), source length (two bytes), width (two bytes) and height (two bytes). Multibyte integers are unsigned big-endian. A 256-byte slot follows, with source bytes first and zero padding. Text dimensions are zero. Image dimensions are 16×16 . No MIME field or payload sidecar is added.

AES-256-GCM (Dworkin, 2007) authenticates and encrypts the entire header and slot using a fresh random key and a unique random 12-byte

A Image → saved UTF-8 → canonical image



B Literal text → saved PNG → literal text



Fixed rank · first frozen case in each direction · both sources recovered byte-for-byte

Figure 1: Actual artifacts selected by the first fixed-rank case in each direction. A transports a canonical 16×16 grayscale array through the displayed verbatim excerpt of a 616-token, 2,827-byte UTF-8 carrier. Recovery restores all 256 canonical pixels, not an arbitrary image file. B shows the complete 63-byte source/recovered messages and actual 32×31 RGB PNG. Pixels use nearest-neighbor display. Each fresh receiver uses its saved carrier, model/profile, shared context and key. The PNG conditioning row is withheld from the carrier and prepended internally. Authentication and separate source-byte equality both pass.

nonce. The associated data are the exact ASCII bytes ImageCalgacus/packet/v1, with no newline. The carried packet is

$$P = \text{nonce} \parallel \text{ciphertext} \parallel \text{tag}, \quad (1)$$

$$|P| = 12 + 264 + 16 = 292 \text{ bytes}.$$

The known 2,336-bit target avoids circular framing around an encrypted length. After authentication the parser validates type, length, dimensions, UTF-8 and all padding bytes, returning no plaintext on failure. The evaluator checks equality separately. Authenticated encryption protects content under its usual assumptions, not the channel's existence.

One immutable packet is shared across the paired methods in a payload/context group. Reusing that ciphertext is not a new encryption under a reused nonce. Keys and nonces come from operating system ran-

domness, independently of sampling seeds. Continuation retains the sealed packet without supplying it to receivers.

3.2 Observable Conditional Distributions

At position, t , the backend supplies eligible symbols E_t , normalized probabilities $q_t(v)$ and a deterministic rank order. Conditioning includes the shared context and all preceding observed symbols, including skips and completion. Text ranks use descending logits, with ascending token identifier breaking exact ties. Image ranks use descending log probability, then ascending channel value. Probability bookkeeping uses float64, stable normalization and ascending-identifier

summation. NaN, positive infinity, empty support and invalid normalization are explicit failures. Numerical zero mass is excluded and the remaining distribution renormalized, without an added probability floor. Ordinary sampling is temperature one inverse CDF sampling in identifier order, with no nucleus truncation.

For text, the model is Llama 3 8B Instruct (Meta, 2024), not an image model. Let T tokenize bytes without BOS insertion or special-token interpretation, and let D return exact bytes without cleanup. Given carrier prefix s , each candidate v must satisfy

$$T(D(s \parallel [v])) = s \parallel [v]. \quad (2)$$

The pool contains the top 256 model tokens *before* filtering. Special or control tokens, empty renderings and invalid UTF-8 are excluded. Equation 2 is checked on the complete carrier prefix, not a prompt/carrier concatenation. The model sees one BOS followed by separately tokenized prompt bytes and carrier identifiers, without a chat template. The receiver tokenizes the delivered file separately, checks exact byte round-trip equality, and replays the same prefix tests. Payload positions, gate skips, arithmetic steps and the 32-token ordinary tail all obey this rule.

For PNG carriers, PixelCNN++ (Salimans et al., 2017) models a native 32×32 RGB canvas. A shared 1×32 RGB row occupies the first row. Only the remaining 31 rows are delivered, giving 992 pixels or 2,976 channel values. Each channel value in $\{0, \dots, 255\}$ is observable. Generation and replay use raster order, with R then G then B within each pixel. The receiver reads an 8bit RGB PNG without palette conversion, alpha compositing or color transformations and prepends the shared row internally.

The network predicts ten mixture weights, channel means, log scales and RGB dependency coefficients per pixel, not 256 independent categorical logits. Let $f_{k,C}$ denote a discretized logistic component mass for channel C . For normalized intensity $z(v) = 2v/255 - 1$, component probabilities are logistic CDF differences over half-bin width $1/255$, with infinite tails at 0 and 255. Log scales are lower bounded by -7 and dependency coefficients use tanh. We evaluate the discrete masses in float64 rather than using the upstream continuous sampler or its small density fallback.

Channels require mixture posteriors, not independent marginals. After red, weights satisfy $w_k^R \propto w_k f_{k,R}(r)$. Green uses normalized w_k^R and mean shift $a_{RGz}(r)$. After green, $w_k^{RG} \propto w_k^R f_{k,G}(g \mid r)$. Blue uses normalized w_k^{RG} with mean shift $a_{RBz}(r) + a_{GBz}(g)$. One network forward per visible pixel supplies all parameters, reused within that pixel. Causal checks establish that unobserved pixels do not alter them.

3.3 Three Packet Coders

Fixed rank. Each packet byte yields its high nibble and then low nibble. A nibble $d \in \{0, \dots, 15\}$ selects eligible rank $d + 1$. Exactly $292 \times 2 = 584$ positions carry the packet. The receiver inverts observed ranks into bytes and stops by this fixed count. Fewer than 16 eligible symbols while the packet is pending causes a support failure, not a smaller alphabet.

Entropy-gated rank. The same four-bit operation is applied only if

$$H(q_t) = - \sum_{v \in E_t} q_t(v) \log_2 q_t(v) > \tau. \quad (3)$$

Otherwise the sender samples ordinarily and consumes no packet bits. The receiver recomputes the strict comparison from the observed prefix, so no gate trace is transmitted. Separate thresholds were calibrated from ordinary development traces and frozen at 0.5983972524635524 bits for text and 3.2159513527579326 bits for channels. They were not adjusted after failures.

Arithmetic A1. We independently implement a finite-precision arithmetic comparator following Neural Linguistic Steganography (Ziegler et al., 2019). A1 identifies our framing adaptation, not a new security construction. It starts with the half-open interval $[0, 2^{32})$. At width R , retain symbols with $q_t(v) \geq 1/R$ and at least the top $\min(2, |E_t|, R)$ symbols. Renormalize retained probabilities, round bin widths to nearest integer with ties to even, truncate before the first cumulative overflow, and assign residual width to the first symbol. Remove zero width bins. Positive widths must sum to R .

The next 32 source bits, zero extended only beyond the packet, select the interval bin. Sender and receiver emit the common leading bits of its lower bound and upper bound minus one, then shift the interval accordingly. A zero bit step updates the interval and consumes carrier capacity. Packet completion occurs at the first common prefix emission reaching 2,336 bits. At most 31 emitted suffix bits are checked to be zero and discarded. There is no end token, final-artifact endpoint dump or implicit flush during ordinary completion. The receiver needs no sender offset to determine progress.

This framing differs from the reference’s final token endpoint handling. A1 can stagnate when a narrow interval straddles a leading bit boundary and quantization retains one bin, leaving that interval unchanged. Its precision and stopping rules remain

fixed. Capacity exhaustion is an outcome, but desynchronization is an implementation defect. An incomplete packet prefix is not authenticated delivery.

3.4 Stopping, Completion and Conditional Inversion

All text carriers have a 2,048 token cap. After packet completion the sender must add exactly 32 ordinary tokens within that cap. A packet completing too late for the tail is packet complete but carrier incomplete. An unfinished packet is not truncated early to reserve a tail. PNG generation always completes all 2,976 channels after the packet stop, including any remainder of a partially used RGB pixel. No packet is stored in PNG metadata or appended bytes.

Inversion follows inductively when both endpoints reconstruct the same distribution and coder transition from each observed prefix. Equation 2 supplies the text invariant, and lossless pixel equality supplies the PNG invariant. The fixed packet size supplies framing. Sufficient support and finite termination remain necessary conditions.

Senders exit before fresh receivers start. Their input directories contain only carrier, profile, context and key, with model assets separately available. Evaluation follows receiver exit. Reports distinguish packet and carrier completion, receiver recovery, authentication, carrier conformance and source equality.

3.5 Photograph-Preserving Rank Embedding

A separate profile, `cover_rank_v1`, carries the same text packet through small changes to a 256×256 RGB8 photograph. For each channel define $B(c) = 4\lfloor c/4 \rfloor$ and require $B(y) = B(x)$ for cover x and carrier y . Both endpoints therefore reconstruct the canonical context $B+2$ without the receiver knowing x . An 8×8 row major grid of 32×32 tiles supplies full parameter maps with 64 GPU forwards. The checkpoint, float32 normalization and CUDA graph settings remain unchanged.

For each selected pixel, enumerate all 64 colors in its three four value channel cells. Their scores are

$$\ell(r, g, b) = \log \sum_k w_k f_{k,R}(r) f_{k,G}(g | r) f_{k,B}(b | r, g). \quad (4)$$

The candidate’s own preceding channels determine green and blue means. Float64 log-domain evaluation retains all 64 candidates, even if exponentiating a score would underflow. Descending scores, with lexicographic RGB ties, define zero-based ranks. Rank

parity gives two 32 color bit classes. The sender chooses the nearest squared-RGB-distance candidate in the required class, with lexicographic distance ties. The receiver reads the observed rank parity. These are coarse context embedding ranks, not the generated mode’s likelihood conditioned on full precision delivered history.

Placement derives a key with HKDF-SHA256 from the run key, salt SHA256 of row-major coarse RGB bytes, and domain-separated info binding the agreed protocol hash. The domain is `ImageCalgacus/cover_rank_v1/placement`, followed by a zero byte and the hash. The hash covers canonical sorted-key JSON of the model and rank rules, excluding arm and runtime paths. HMAC-SHA256 priorities use `pixel`, a zero byte and each four-byte big-endian raster index. Sorting priorities, then indices for ties, selects 2,336 distinct positions in packet-bit order, most significant bit first. The hidden nonce is not a placement input. The paired model free baseline labels the identical candidate cube by $(r + g + b) \bmod 2$, using the same positions, sealed packet and nearest-distance rule.

The receiver receives only PNG, profile/arm and key. Cell invariance yields the same tiles, selected positions and rank map, so observed parity inverts every bit before authenticated parsing. Each channel changes by at most three. With only 2,336 selected pixels, pooled RGB MSE is at most $9(2336)/65536$, giving the conservative theoretical bound $\text{PSNR} \geq 53.07$ dB. Authentication protects the recovered packet, not every unused cover pixel.

4 EXPERIMENTAL DESIGN

4.1 Sources, Contexts and Pairing

Development used 20 images and 20 texts with two contexts per direction, allocating 80 fixed rank units, 40 static text comparisons and 32 gated/arithmetic timing units. Five synthetic image fixtures were supplemented by 15 Fashion-MNIST training images. Five literal text fixtures, including non-ASCII UTF-8, were supplemented by 15 narrative spans from Gutenberg ebooks 11 and 84. Sources, canonicalization, contexts, profiles and thresholds were frozen before prospective generation.

The held out images are 20 Fashion-MNIST test samples, two per class (Xiao et al., 2017), BOX resized from 28×28 to 16×16 grayscale. Twenty nonoverlapping Gutenberg spans from Austen’s *Pride and Prejudice* and Doyle’s *The Adventures of Sherlock Holmes* occupy the 32to64 and 65to128byte

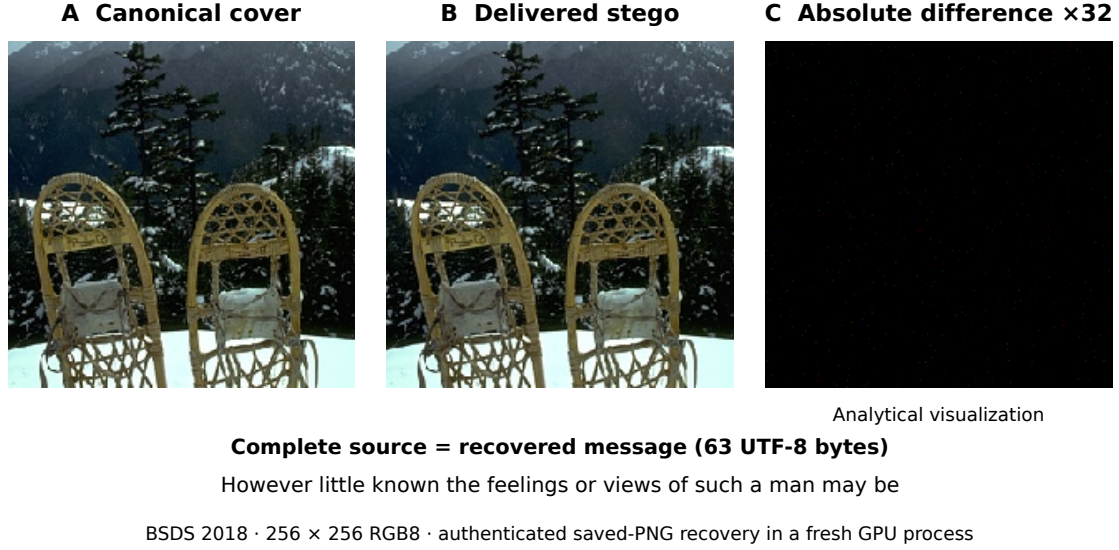


Figure 2: Complete photograph transport for the same predetermined BSDS500 test case, 2018 (Arbeláez et al., 2011). A is the canonical cover and B the actual delivered model-rank PNG, both 256×256 RGB8 at identical display scale without enhancement. C is an analytical visualization, $32|B - A|$ per RGB channel, not a carrier transformation. Black denotes no change. The shared text annotation gives the entire source and exactly recovered message. A fresh GPU receiver reconstructs the coarse context and ranks from B using only its profile and key. Authentication and independent byte equality pass.

bands, ten each. Lengths are 58to128 bytes, mean 93.2. Frozen hashes, indices/offsets and exclusions prevent duplicate or development overlap without inspecting outcomes. Fashion MNIST’s MIT notice and Gutenberg attribution are retained. Text copyright varies by jurisdiction.

The prospective prompt is "Write a calm field-journal entry about a walk through a temperate forest. Use continuous prose." Row1 is the first row of an independently generated development image with context seed 2001. Neither context is included in the delivered carrier. Table 1 summarizes the frozen allocation.

The allocation is $20 \times 3 \times 2 = 120$ stego units. Each group shares one immutable packet, source and model across methods. Terminal failures remain observations, without duplicate credits or replacements.

One independent ordinary trace serves each group. Text traces reach 2,048 tokens, their longest realized carrier, with matched prefixes for fixed/gated methods. PNG methods share full images. Thus 120 associations represent 40 controls. The frozen rule assigns duplicate artifacts to the lowest ordered eligible group. No prospective duplicates or unscorable carriers occurred.

4.2 GPU Execution and Timing

All inference runs serially on one NVIDIA RTX 5000 Ada Generation GPU with 32 GB memory and driver 590.48.01. Text uses the QuantFactory Llama 3 8B Instruct Q4_K_M GGUF and embedded tokenizer. Explicit `n_gpu_layers=-1` is verified as 33/33 eligible layers offloaded, with `logits_all=True`, eight threads, a 4,096-token context and batch/microbatch one. Context/KV state is reset between sequences and evaluation is incremental within a sequence. No CPU or partial-offload fallback is permitted.

PixelCNN++ uses a pretrained CIFAR-10 checkpoint with five residual blocks, 160 filters and ten mixtures. Strict loading, CUDA parameter/input placement and evaluation/inference modes are verified. The float32 network uses neither TF32 nor mixed precision. Deterministic algorithms, disabled cuDNN benchmarking, synchronous launches and fixed cuBLAS workspace are retained. The pinned SHA256 identifiers begin 86c8ea6c8b755687 for the GGUF and a5ed558f6d4098ce for PixelCNN++. The latter checkpoint is `pcnn_lr.0.00040_nr-resnet5_nr-filters160_889.pth`, loaded through the PyTorch port at revision 7cb4436. Image execution uses CUDA 12.4, cuDNN 90100 and `CUBLAS_WORKSPACE_CONFIG=:4096:8`. Text disables `llama.cpp graphs/fusion`. Both backends have positive process-specific GPU activity evidence.

Table 1: Original generated carrier specification. Dimensions are width by height. Both directions share a 292byte packet, but carrier symbols and useful rate units differ. One sealed packet is shared across methods within each payload/context group.

Property	Image payload in text	Text payload in PNG
Canonical source	16 × 16 grayscale, 256 raw bytes	Literal UTF-8, 32–128 bytes
Delivered artifact	Strict UTF-8, at most 2,048 tokens	32 × 31 RGB8 PNG, 2,976 channels
Neural model	Llama 3 8B Instruct, Q4_K_M GGUF	CIFAR-10 pretrained PixelCNN++
Runtime	llama-cpp-python 0.3.23, 33/33 CUDA layers	PyTorch 2.5.1+cu124, float32 CUDA
Shared context	Forest prompt, separate tokenization	Private 1 × 32 RGB row, 96 bytes
Eligible symbols	Top 256, then complete-prefix filtering	Discrete R/G/B values 0–255
Coders	Fixed radix 16, gated radix 16, arithmetic A1	Same coders and 2,336-bit target
Completion	32 ordinary tokens after the packet	All channels through the last visible pixel
Payload groups	20 held-out images, one context	20 held-out texts, one context
Controls	20 traces, matched prefixes	20 PNGs shared across methods

Prospective timings use the reference image path, not the later graph optimization. Charged latency spans child process launch to exit, including imports, hashing/loading, occupied CPU work, inference, serialization and teardown. Encode/decode phase times exclude some overhead. CUDA events are nested within phases and are not added again. Fresh loading does not imply eviction of filesystem caches. The separate evaluator and wrapper monitoring joins are outside this boundary.

4.3 Metrics and Grouped Uncertainty

For L source bytes and N delivered symbols, exact goodput is $G = 8L/N$ only for an authenticated, exact and conforming recovery, and zero otherwise. Text uses bits/token and PNG bits/channel value. Multiplying the latter by three gives bits/pixel. Complete-packet rate counts 2,336 bits through the packet stop or across the full carrier. It is undefined for incomplete A1 text, whose prefix progress is reported separately.

Recovery uses two-sided 95% Clopper–Pearson intervals, [0.8316, 1] for 20/20 and [0, 0.1684] for 0/20. These are Bernoulli working model bounds for a small fixed allocation. Continuous quantities, AUCs and paired differences use 2,000 frozen PCG64 draws, base seed 2026090901 plus direction offset. Whole groups are resampled within image class or text-length band, carrying all methods and shared controls together. Intervals use the 2.5th and 97.5th percentiles. Boundary paired binary comparisons additionally retain conservative bounds from 97.5% marginal Clopper–Pearson intervals, rather than interpreting collapsed bootstrap intervals as certainty.

4.4 Observer and Context Mismatch

The observer knows the exact model, profile, eligibility and the scoring rules and, initially, the correct

context, but not the key or source. Its a frozen whole carrier scores are

$$S(y; c) = -N^{-1} \sum_t \log_2 q_t(y_t), \quad (5)$$

$$R(y; c) = N^{-1} \sum_t \log_2 \text{rank}_t(y_t). \quad (6)$$

Higher always indicates stego. AUC is the probability that stego score exceeds an ordinary score, with a half weight for ties. No score orientation or threshold is selected using held out labels. Scorable failed carriers remain included.

The extension retains correct row scores and rescores the same 60 stego PNGs and 20 shared controls under row2. This random RGB development row was frozen using PCG64 seed 2002, not detection outcomes. The scorer reads only PNG, profile and row and evaluates every delivered channel. Three predetermined development artifacts, fixed, A1 and ordinary PNG, first reproduced their retained scores and both available probability stream digests. Paired AUC changes use 2,000 length stratified draws, seed 2026090902, carrying both conditions, all methods and shared controls together. They are repeated measurements, not new transmissions. Within artifact score shifts are descriptive and post hoc, with no added tests or intervals.

4.5 Matched Development Benchmark

The lowest ordered development payload in each of three byte bands gives 32, 96 and 128 byte messages under row1. Fixed rank has the three matched timing repetitions per payload with alternating reference/graph order. The 32 byte case also tests gated and A1 once, giving 11 comparisons and 22 recoveries, and not 11 independent payloads.

The graph path captures the unchanged PixelCNN++ forward after three warmups and replays from persistent input/output buffers. Both modes use fresh senders and receivers and the same retained

packet. Equivalence requires exact eligible identifiers, ordering, float64 probabilities, PNG bytes, recovered bytes and completion outcomes. Headline speedup divides matched mean charged pair times over the nine fixed comparisons. Throughput pools recovered source bytes over both processes. No numerical tolerance is introduced.

4.6 Bounded Photograph Evaluation

The photograph allocation is separate from the 120 generated carrier transmissions. We selected the six lowest numeric BSDS500 training identifiers and twenty lowest test identifiers (Arbeláez et al., 2011), converted them to RGB and took deterministic 256×256 center crops without resizing. They pair in order with six retained development texts and the twenty prospective text sources. Both arms use one sealed packet per cover/text group. After twelve development recoveries and focused negative/lossless checks, the unchanged implementation was frozen for forty evaluation carriers.

Fidelity is relative to canonical covers. PSNR uses pooled RGB MSE and peak 255. SSIM averages channels with an 11×11 Gaussian window, $\sigma = 1.5$, valid centers, population covariance, $K_1 = .01$ and $K_2 = .03$. We also record changed pixels/channels, maximum change, PNG bytes and fresh-process costs. Partition disagreement is the smaller of a label Hamming distance and its complement, divided by 64. Summaries are descriptive paired measurements, and without a trained detector or new inferential tests.

5 RESULTS

5.1 Exact Artifact Recovery and Useful Rate

The prospective allocation completed all 120 stego outcomes and 40 independent controls. There were 100 exact, authenticated and conforming recoveries and 20 retained A1 text capacity failures. All 280 model processes have GPU execution evidence. No source replacement, protocol retry, timeout or infrastructure failure was needed. Table 2 and Figure 3 retain every attempt.¹

Fixed and gated coding recover 20/20 payloads in each direction. Fixed text uses 584 packet positions and 32 completion tokens, giving 3.32468 useful bits/token. Gating adds a mean 39.8 skips, reducing

goodput by 0.2000 bits/token [0.1805, 0.2192] and increasing charged pair time by 23.96 seconds [20.34, 27.58]. Whole-carrier mean surprisal decreases by 0.3912 bits/token [0.3079, 0.4764].

All PNG methods recover 20/20, giving mean exact goodput of 0.25054 bits/channel [0.24731, 0.25349], or 0.75161 bits/pixel. Equal rates follow partly from the fixed full canvas and do not imply equal coder capacity. Fixed uses 584 packet channels, gating adds a mean 100.5 skips, and A1's mean packet span is 597.85 channels, including 89.65 zero bit positions. The ordinary completion fills the remaining channels.

Framing/encryption adds 36 bytes. Texts leave a mean 162.8 padding bytes. Images fill the slot. Including completion, packet transport is 3.79221 bits/token for fixed text, mean 3.56404 for gated text, and 0.78495 bits/channel for every PNG method. A1 text has no complete-packet rate. Its PNG termination position is already included in the packet span, not an extra position.

5.2 Arithmetic Text Reaches the Capacity Boundary

All 20 A1 text streams reach the 2,048 token cap before producing the 2,336bit packet. They recover between 581 and 2,216 packet-prefix bits, with mean 1,456.85. None reaches zero extension or finite-message termination. Accordingly, none authenticates or delivers useful source bytes, despite the availability of partial information. Mean charged 'encode plus decode' time is 826.57 seconds, over six times fixed text, and all that cost remains in the results.

Independent interval, partition and sender/receiver prefix checks agree with all 40 prospective arithmetic audits. Every arithmetic PNG terminates and recovers. Two failed texts have one and three isolated unchanged steps, with longest consecutive run one. A successful PNG also has one. These do not support sustained stagnation as the text failure cause. Zero-bit steps can continue narrowing an interval without emitting a leading bit.

Cumulative surprisal closely tracks recovered text prefix information, supporting low information under the cap rather than decoder disagreement or an unexecuted flush. This observed limit is distinct from A1's separately demonstrated stagnation case.

5.3 Detectability with the Correct Context

Correct context scores show that exact recovery and low detectability are different endpoints. Both fixed

¹Drafting used Codex (OpenAI, 2025).

Table 2: Prospective outcomes and original reference-path runtime. Each cell has 20 attempted payload groups. Exact recovery is 20/20 with 95% interval [83.16%, 100%], or 0/20 with [0%, 16.84%]. Phase means exclude loading and some process overhead. Charged means include both fresh processes and unsuccessful attempts. Goodput units are bits/token for text and bits/channel value for PNG.

Carrier	Method	Exact	Goodput	Encode (s)	Decode (s)	Charged (s)	Failure
UTF-8	Fixed	20/20	3.3247	45.63	45.31	123.33	None
UTF-8	Gated	20/20	3.1246	57.31	57.30	147.29	None
UTF-8	A1	0/20	0.0000	398.15	396.01	826.57	Capacity
PNG	Fixed	20/20	0.2505	85.27	85.31	178.18	None
PNG	Gated	20/20	0.2505	85.29	85.65	178.53	None
PNG	A1	20/20	0.2505	85.59	85.77	179.01	None

and gated text have empirical mean surprisal AUC 1.000, with bootstrap interval [1.000, 1.000]. Every observed stego score exceeds the corresponding ordinary sample range. This is complete empirical separation under the tested observer, not an assurance of perfect future detection. Failed A1 text gives AUC 0.380 [0.260, 0.503]. The below chance orientation is retained rather than reversed to improve a detector result.

PNG scores are less separated. Fixed mean surprisal AUC is 0.8850 [0.7425, 0.9850], gating 0.8300 [0.7000, 0.9375], and A1 0.5575 [0.3750, 0.7426]. Arithmetic PNG combines exact delivery with lower observed separation for this score, although its interval spans a wide range around chance. Every cell has 20 scorable stego artifacts and 20 unique matched controls, without exclusions. The secondary log-rank AUCs are 1.000, 1.000 and 0.360 [0.2425, 0.4801] for fixed, gated and A1 text, respectively. PNG values are 0.8400 [0.6825, 0.9650], 0.7700 [0.6299, 0.8975] and 0.5750 [0.4050, 0.7576].

5.4 PNG Detection Depends on the Tested Context

All 80 retained PNGs are fully scorable under row2. Fixed rank primary AUC decreases from 0.8850 to 0.6975, a paired change of -0.1875 with interval $[-0.2775, -0.1000]$ (Figure 4). Gating changes from 0.8300 to 0.7475, or -0.0825 $[-0.1625, 0.0100]$. A1 changes from 0.5575 to 0.5925, or $+0.0350$ $[-0.0225, 0.1050]$. The fixed reduction is resolved by its paired interval. The gated and A1 changes remain uncertain.

Under mismatch, the primary fixed interval [0.5150, 0.8700] only narrowly excludes 0.5, and its secondary rank-score AUC is 0.6625 [0.4725, 0.8475]. The primary gated interval [0.6074, 0.8825] remains above chance. Thus this mismatch weakens the tested fixed score without removing all information from mean-surprisal detection. A resolved change for one method and an uncertain change for

another do not themselves establish different method sensitivities.

Mean 'row2 minus row1' score changes are 0.610 bits/channel for controls, 0.334 for fixed, 0.513 for gated and 0.629 for A1. These descriptive shifts narrow the paired fixed-minus-control gap from 0.508 to 0.231. Larger control increases are consistent with, but do not explain, the AUC change. Row1 comes from the generated image and row2 from random RGB values, so the context correctness is not isolated from row statistics.

5.5 Exact GPU Replay Improves Useful Throughput

CUDA graph replay reduces mean fixed rank charged encoding from 88.95 to 17.36 seconds and decoding from 88.87 to 17.28 seconds. Combined latency falls from 177.8186 to 34.6417 seconds, a matched 5.133 fold improvement. Pooled useful throughput rises from 0.4799 to 2.4633 recovered bytes per second. These nine fixed timing comparisons include both fresh processes. Gated and A1 single payload combined times change from 180.25 to 34.61 and 182.85 to 35.11 seconds.

All 11 reference/graph comparisons preserve exact eligible identifiers, rank ordering and float64 probability streams at all 2,976 channels. All 22 PNGs recover identical source bytes, without numerical tolerance or packet rerolls. Graph replay therefore remains an explicit optional mode on this stack. It does not retroactively alter prospective timings. Peak allocated memory is 484.479 versus 473.849 MiB, reserved memory 516 versus 744 MiB and sampled process memory 912 versus 1,176 MiB for reference versus graph. These measures overlap and are not additive. PID-matched GPU activity samples are intermittent, not a sustained utilization estimate.

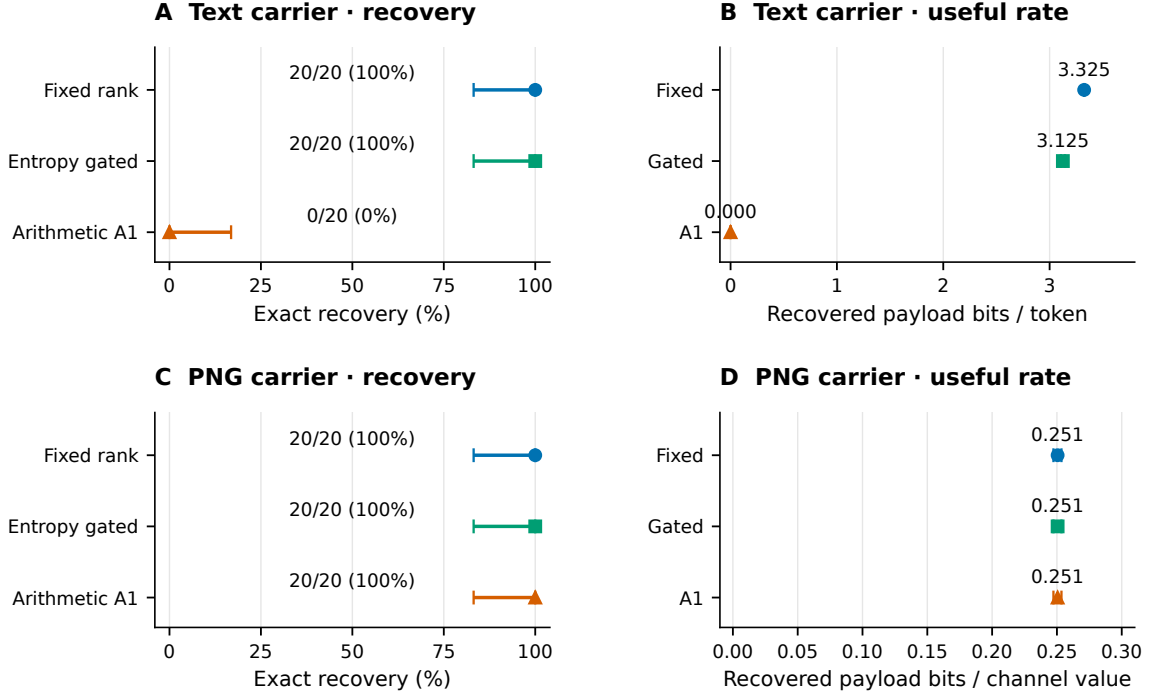


Figure 3: Held out recovery and exact useful rate for 20 payload groups per direction. Recovery whiskers are the accepted 95% Clopper-Pearson bounds. Rate whiskers are the accepted stratified group-bootstrap intervals, keeping paired methods together. All failed A1 text transmissions contribute zero goodput. Text and PNG rate axes use different carrier symbols. Zero-width observed rate intervals do not imply universal reliability.

5.6 Photographs Retain Fidelity and Exact Recovery

Both arms recover 6/6 development and 20/20 held out messages from saved PNGs, with authenticated source equality and no replacements or retries. Model ranks reproduce exactly across fresh processes. Development wrong-key, bit-change and lossless checks pass. Figure 2 shows the first held out photograph, distinct from generated-image mode (Figure 1B).

The model arm averages 70.255 dB PSNR and 0.9999664 SSIM, with a maximum observed channel change of two. Parity’s maximum is one. The model arm changes 1.781% of pixels and 0.613% of channels. The packet rate is $2336/65536 = 0.0356445$ bits/pixel, with 58to128 useful bytes. A fresh process encoding and decoding average 6.998 and 6.884 seconds. These costs concern fixed context tile evaluation, not a speedup comparison with autoregressive generation.

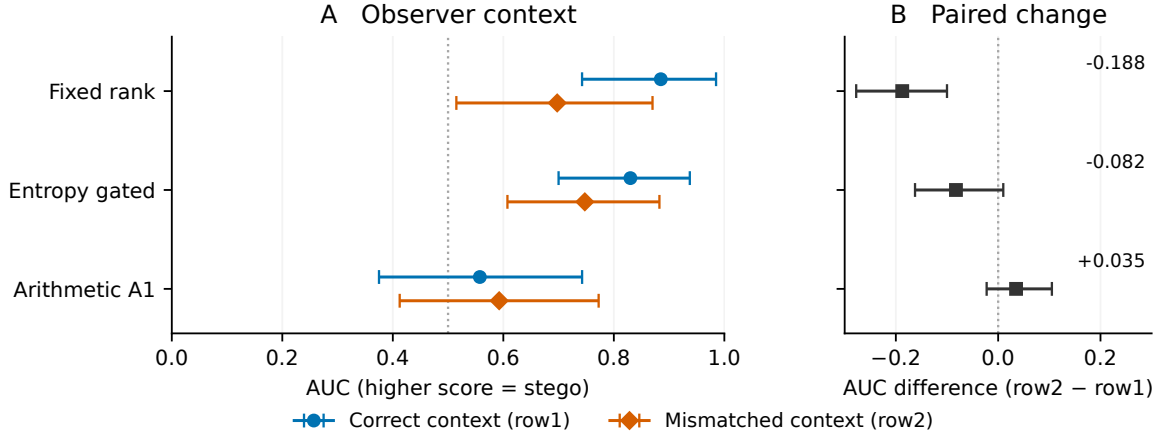
All 46,720 selected held out pixel partitions differ from parity even after allowing a bit-label swap. Mean normalized disagreement is 0.44844. Parity also recovers 20/20, with mean 70.398 dB PSNR and

0.9999688 SSIM (Table 3). Its mean paired process cost is 1.509 seconds, compared with 13.882 for model ranks. The paired ‘model minus parity’ PSNR difference is -0.144 dB. Thus distinct learned partitions provide no demonstrated recovery, fidelity or runtime advantage in this comparison. The detection experiments concern generated PNGs, not these photographs.

6 DISCUSSION AND LIMITATIONS

The three transport paths expose different constraints. Generated text requires serialization consistency and sufficient information along the realized trajectory. Generated PNGs provide directly observable channel symbols. Photographs instead preserve a reconstructible coarse context. Their learned partitions differ from RGB parity, but model likelihood is not a distortion objective. The simple comparator’s favorable fidelity and cost show that partition distinctness alone is not an embedding advantage.

Development isolates saved text serialization.



Same 80 PNGs; 20 payload groups; mean surprisal; paired 95% grouped intervals

Figure 4: Primary PNG detection with correct row1 and mismatched row2. The same 60 stego PNGs and 20 shared ordinary controls appear in both conditions. Panel B gives paired AUC change, row2 minus row1. Whiskers use the accepted 2,000 length-stratified group draws, jointly retaining observer conditions, methods and control ownership. The observer still knows the exact model and procedure. Row2 is the previously frozen random-RGB row, not a row selected for low detection. Reference lines are not fitted classification thresholds.

Table 3: Photograph evaluation, twenty paired held out covers per arm. Fidelity and timings are descriptive means. Times include fresh-process startup, loading, graph setup and teardown. The parity baseline has no neural inference. Both arms share each group’s packet and positions.

Arm	Exact	PSNR (dB)	SSIM	PNG bytes	Encode (s)	Decode (s)
Model ranks	20/20	70.255	0.9999664	116,772.85	6.998	6.884
Simple parity	20/20	70.398	0.9999688	116,762.50	0.823	0.686

Twenty image payloads under two prompts give 40 packet-paired cases. Complete-prefix filtering recovers 40/40, versus 22/40 for singleton filtering (8/20 forest, 14/20 soup). All 18 drift failures remain unrepaired. This supports the consistency requirement alongside published stepwise verification. The independent sample comprises 20 payloads, not 40. With twenty generated PNG resaves also recover exactly. JPEG, cropping and paraphrasing remain untested.

The prospective study has 20 held out groups per direction and two pinned models. Shared controls and rescoring retain their dependencies. Near chance AUC and context dependence establish neither cryptographic secrecy nor resistance to arbitrary observers. Photograph fidelity is likewise not a detection evaluation. A1’s capacity and finite-precision limitations concern this adaptation rather than arithmetic coding generally.

Graph replay reduces image-model launch overhead. Text filtering still consumes approximately 71%, 75% and 89% of fixed, gated and A1 encoding time. Faster inference cannot remove that CPU bottleneck or supply missing information under the

frozen cap. The matched GPU measurements characterize three development payloads on the stated stack, separately from the photograph tile workload.

7 CONCLUSION

We develop a shared authenticated protocol for exact image transport through generated text and exact text transport through generated PNGs or bounded photograph changes. Fixed and gated coding recover all prospective payloads in both generated directions. Arithmetic A1 recovers PNG payloads but reaches the text cap. Context mismatch changes measured PNG detectability, and matched CUDA graph replay improves throughput by 5.13 times with exact checked probability and artifact equivalence.

Invariant cover pixels enable reproducible model-rank embedding and exact held-out photograph recovery. The simple parity attains the same recovery with slightly better fidelity and lower cost. With combined evaluation makes saved-artifact recovery, observer knowledge and execution cost explicit, sepa-

rately measurable endpoints.

AI ASSISTANCE DISCLOSURE

OpenAI Codex assisted with implementation, analysis scripts, and figure preparation (OpenAI, 2025).

REFERENCES

- Arbeláez, P., Maire, M., Fowlkes, C., and Malik, J. (2011). Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916. <https://doi.org/10.1109/TPAMI.2010.161>.
- Chen, B. and Wornell, G. W. (2000). Preprocessed and post-processed quantization index modulation methods for digital watermarking. In *Security and Watermarking of Multimedia Contents II*, volume 3971, pages 48–59. SPIE. <https://dsp-group.mit.edu/wp-content/uploads/2024/11/Preprocessedandpost.pdf>.
- Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. Technical Report SP 800-38D, National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>.
- Filler, T., Judas, J., and Fridrich, J. (2011). Minimizing additive distortion in steganography using syndrome-trellis codes. *IEEE Transactions on Information Forensics and Security*, 6(3):920–935. <https://doi.org/10.1109/TIFS.2011.2134094>.
- Gehrmann, S., Strobelt, H., and Rush, A. M. (2019). GLTR: Statistical detection and visualization of generated text. In *Proceedings of ACL: System Demonstrations*, pages 111–116. Association for Computational Linguistics. <https://doi.org/10.18653/v1/P19-3019>.
- Mantzaris, A. V. (2026a). LlmStenoExplore. Software repository and empirical exploration. Revision b9f1650. <https://github.com/mantzaris/LlmStenoExplore>.
- Mantzaris, A. V. (2026b). RankCloak conceals the surface form of synthetic cryptographic artifacts in language model generated text. Repository manuscript and implementation. Revision ce853d4. <https://github.com/mantzaris/llm-rankcloak>.
- Meta (2024). Meta Llama 3 model card. Official model documentation. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- Norelli, A. and Bronstein, M. (2026). LLMs can hide text in other text of the same length. arXiv:2510.20075v6. Original preprint 2025.
- OpenAI (2025). Introducing Codex. Official product description, 16 May. <https://openai.com/index/introducing-codex/>. Tool attribution, not a scientific evidence source.
- PyTorch Contributors (2024). CUDA semantics. PyTorch 2.5 documentation, CUDA graphs. <https://github.com/pytorch/pytorch/blob/v2.5.1/docs/source/notes/cuda.rst>.
- Salimans, T., Karpathy, A., Chen, X., and Kingma, D. P. (2017). PixelCNN++: Improving the PixelCNN with discretized logistic mixture likelihood and other modifications. arXiv:1701.05517.
- Schroeder de Witt, C., Sokota, S., Kolter, J. Z., Foerster, J., and Strohmeier, M. (2023). Perfectly secure steganography using minimum entropy coupling. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.14889>.
- Tancik, M., Mildenhall, B., and Ng, R. (2020). StegaStamp: Invisible hyperlinks in physical photographs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. <https://arxiv.org/abs/1904.05343>.
- Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747.
- Yan, R. and Murawaki, Y. (2025). Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of EMNLP*, pages 7076–7098. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.emnlp-main.361>.
- Yan, R. and Murawaki, Y. (2026). Efficient provably secure linguistic steganography via range coding. In *Proceedings of ACL (Volume 1: Long Papers)*, pages 890–907. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.acl-long.39>.
- Zhang, S., Yang, Z., Tu, H., Yang, J., and Huang, Y. (2021). Pixel-Stega: Generative image steganography based on autoregressive models. arXiv:2112.10945.
- Zhu, J., Kaplan, R., Johnson, J., and Li, F.-F. (2018). HiDDeN: Hiding data with deep networks. arXiv:1807.09937.
- Ziegler, Z. M., Deng, Y., and Rush, A. M. (2019). Neural linguistic steganography. In *Proceedings of EMNLP-IJCNLP*, pages 1210–1215. Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1115>.